

Objectively evaluating one's own design is impossible. It is often easy to miss flaws in usability when you are the one who designed the system, and you innately know how it works. That being said, I am going to attempt to create as close to an accurate evaluation as possible using two different evaluation methods. First, I am going to use a quantitative approach and time how fast I can complete the ticketing process on my ticketing application. I will then compare this to how quickly I can complete a booking on a pre-existing web-based plane ticketing service, that being booking.com. I will spend most of my time attempting to do an analysis using the information I learned in class to emulate an expert review of the program.

I am going to reiterate the main goals of my ticketing service before getting into the evaluation process so that I have them to compare and analyze the results of each method. The most important principle my service must adhere to is universal usability (at least within my target user group). My target user base is inexperienced, rural users, specifically in the United States (or other English-speaking countries). The most important factors of this userbase that I considered while designing the service are that they are inexperienced with technology and that they are easily frustrated. To mitigate these two issues, I have attempted to design my ticketing application to be as simple and streamlined as possible. To achieve this, I attempted to include as little information as possible during each step in the ticketing process. I also did my best to explain every step and include good, consistent labeling to keep the users informed and allow them to feel like they know what they need to do. Finally, to mitigate frustration, I included ways for the users to backtrack and go over the previous steps at each point in the process. Keeping this in mind, I will now move on with the evaluation.

First, I will show the results of the quantitative timing of the process. To try to eliminate discrepancies caused by me being more familiar with my program and thus knowing how to complete it more quickly, I practiced the booking process on Booking.com. Completing the booking process on Booking.com took me 1 minute and 54 seconds. Completing the booking process on my own application took 45 seconds. There are a few external factors that can account for this difference outside of usability that I have to mention. I attempted to pause and un-pause the timer while using Booking.com because the website took a second or two to load while moving between pages. My application is locally stored on my device, so it loads nearly instantaneously. Additionally, booking.com requires the user to enter in their personal information before purchasing the ticket while mine does not. Both of these together likely added at most 30 seconds which means my service was still about 45 seconds faster (which is about half the time that Booking.com took). This quantitative timing does seem quite good for my ticketing process. One of my goals was to keep the service simple and streamlined and this is a good indicator that I was able to do that. Being

shorter while going through the same process means that there was less overall thinking on each page. All of the information required to move on was present front and center, so the user didn't have to search for it. On Booking.com, I had to scroll through pages at various points as the website tried to sell me things like travel insurance and seat upgrades. By avoiding that, my program is able to streamline the process which I hope will make it more usable for inexperienced users. The quantitative method does bode well for my service.

Finally, I am going to do an expert review, focusing on how easy learning and using the program would be for an inexperienced user. To do this, I will go through each step in the process and highlight all of the potential issues that I can see as well as anything that shines as particularly useful in streamlining the application, reducing frustration, and promoting universal usability.

There is nothing remarkable to note about the login page so I will skip ahead. I will also skip the options page as there is nothing particularly unique or complicated about that page.

On the home page there is a split presented to the user on whether they want to use an AI Chatbot or just fill in a form to find their flight. Including the option to choose between the two was intended to protect the internal locus of control of more experienced users by allowing them to skip chatting with the bot. I do think, however, that this decision might actually have been incorrect. My target demographic might get confused having two different options to choose from. Inexperienced users might not know the difference between the chatbot and form and making that decision could lead to frustration. In a future implementation, not including the option and just having one path that combines the two approaches (perhaps an experienced user could ask the chatbot to give them a form to fill out rather than using natural language) would help strengthen the application's accessibility within the inexperienced user base. Additionally, uploading the file to display previous flights is not very accessible to inexperienced users. I was not able to find a way to automatically do this without using a server and I don't have any experience implementing servers, so I wasn't able to make this work. In future versions of the application, this problem would be solved by having user account details saved on a server.

The next two pages are parallel to each other. I will discuss the chatbot page first. I believe this page is successful in catering to my target user group. It is very simple, presenting itself how any texting or messaging service is presented. Additionally, the chatbot being there to explain things to the user helps answer the many questions that a new user might have, reducing frustration at using the application. I did not have time to implement the AI itself, so the current page just has a fill-in-the-blank on a prewritten message to fill in

details. This would not be the case in an actual implementation. This page is widely successful in helping solve the problems that my userbase might present.

The form-fill-in page is the other option the user can take. This page is also very simple, presenting just a few options. Having the ability to pick the travel dates off of a calendar is very helpful for inexperienced users as a calendar offers a familiar metaphor for the user that would allow them to easily pick their preferred date. The dropdowns are a little bit less successful for this. There is a lot of information presented to the user, and they must know what airport is closer to them in order to select it. It can be difficult finding their desired start and destination from this dropdown. It would be much better for all users if it was in alphabetical order. That would go a long way towards promoting easy use and helping mitigate the frustration that could come from not being able to find your preferred airport. Otherwise, there is nothing particularly outstanding or frustrating about this page.

The next page is the flight selection page. It shows the user a few options of flights they could choose and allows them to pick them. I do think this page is very successful. All of the categories in the flight table are descriptive and the buttons being in each row make for an easily understandable path forward. These buttons are also consistent with the progression buttons on the form page and the AI page so it should be even easier for a user to know how to navigate without relying on short-term memory as much.

The next page is the flight details and payment page. This page has a couple of issues. You have to scroll to complete this page which is something I wanted to avoid. Perhaps splitting the flight details page and the payment page into two separate pages might help mitigate this problem. Needing to scroll reduces how streamlined the app is and avoiding it completely would be ideal. Additionally, the seat selection UI is not as effective as I would like it to be. There isn't any distinction between the rows. With some more practice with CSS and HTML I do believe I could fix this issue.

Finally, the confirmation page is there to provide a sense of closure to the user. I believe it is successful, however, I do think that the page could potentially use some stylistic changes to help make it more aesthetically pleasing. The black text on a white background is fine but it isn't very exciting, and many people aren't fans of white backgrounds on websites these days. Other than that, the page is pretty unremarkable. It is successful in providing closure.

For a few general notes, the back buttons throughout the ticketing process are successful in allowing easy reversal of actions and helping to reduce the frustration that mistakes can cause. I also believe that the consistent design layout and color pallet used throughout the

application will help my target user group stay more comfortable throughout the ticketing process.

In general, based on both review methods that I used, I think that my application is fairly good. I was very successful at keeping it simple and reducing frustration and mostly successful in keeping it streamlined, which were my main goals. Hopefully this will allow the application to resonate with my target audience better. I didn't have time to implement an actual AI chatbot or server structure which is disappointing but the potential they have to improve the functionality of the application is huge. I would evaluate my application as successful in maintaining usability by an inexperienced user base which was my goal.